Design and FPGA Implementation of an Efficient 8×8 Multiplier Using the Principle of Vedic Mathematics



Smitha Bhat Kaje and Jagadish Nayak

Abstract This paper is regarding the design and analysis of an efficient 8×8 Vedic multiplier by the principle of Vedic mathematics [1]. Here, a unique Vedic multiplier architecture is adapted, which is not based on the regular method of multiplication (addition, shifting). The design is as per the "Urdhwa-Tiryakbhyam Sutra" of Vedic mathematics. Two numbers (binary of 8-bit each) are multiplied using the methodology of this principle/sutra. "Urdhwa-Tiryakbhyam" means "vertically and crosswise", wherein the partial products are computed at once, thus lessening the delay and hence making the multiplication faster. This 8 by 8 bit multiplier is coded in Verilog HDL and tested on a DE10-lite FPGA kit.

Keywords Vedic maths · Principles · Urdhwa-tiryakbhyam sutra · Verilog HDL · DE10-lite FPGA

1 Introduction

Multiplication is an operation much needed in various applications. A multiplier is a prominent block in the processing systems. Multipliers are vital components for computation purpose in several high-performance systems such as microprocessors. Faster the multiplier, the better is the system's performance. Usually, the multiplier is the part of the system that takes most computational time. There is a requirement of faster multiplier due to the domination of multiplication in the execution time of most of the applications. Hence optimizing in terms of delay and speed of the multiplier is an important requirement in designing.

DE10-lite has an efficient platform for hardware design, which is setup on the Altera MAX 10 FPGA. The inputs are entered through the DIP switches, and the LEDs and the seven segment display are used to show the outputs. The results of the

S. B. Kaje (✉)
M E Microelectronics, BITS Pilani, Dubai, UAE
e-mail: kajesmita@gmail.com

J. Nayak
HOD, EEE Department, BITS Pilani, Dubai, UAE

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2021

C. R. Panigrahi et al. (eds.), *Progress in Advanced Computing and Intelligent Engineering*, Advances in Intelligent Systems and Computing 1299,
https://doi.org/10.1007/978-981-33-4299-6_61

Vedic multiplier [1] and a regular Booth's multiplier [4] are tabulated and compared for performance in terms of factors like the logic elements used and propagation delay. This architecture can be further developed to multiply complex numbers with high speed and reduced delay. This project's intention is to increase the recognition of Vedic mathematics in the field of engineering and to get considered in the design of components in commercial products of everyday use.

2 Literature Survey

In the literature, there have been various algorithms proposed to perform the multiplication. All of them have their own advantages and provide trade-offs with respect to speed, delay, circuits complication, area, and power. The literature survey of the references used in this paper is as follows.

Swami [1] researched for eight years on Vedas and has written a book on Vedic mathematics. He bi-furcated the mathematics into 16 simple sutras (formulae) and 16 sub-sutras which are the backbones of Vedic mathematics. The greatest advantage of Vedic mathematics is that it converts the complex calculations of traditional mathematics into simple ones, thereby optimizing processing time, area, and delay. This book gives an insight on all these principles in a detailed manner.

In John et al. [2] described how to reduce the delay and consumed power in multipliers, by the carry select addition technique. Nine full adders and a special 4-bit adder are used to design a 4×4 Vedic multiplier, resulting in reduced delay. Subsequently, an 8-bit multiplier is formed using four 4-bit multipliers with three-carry select adders.

In this paper, Gadakh et al. [3] described about the design of up to 32-bit Vedic multiplier that reduces power, delay, etc. Xilinx 14.7 software is used for simulation. The results are compared for various Vedic multipliers of different architectures.

In Sunitha et al. [4] used four 4X4 multipliers, designed using Urdhwa-Tiryakbhyam sutra for generating partial products. Optimization in terms of power and delay is achieved by using high speed 4:2 compressor architecture. Three modified ripple carry adders are used for accomplishing the addition of the partial products.

In Ingle et al. [5] compared Vedic multipliers with conventional multipliers like Booth's multiplier. Multipliers are coded in Verilog, and simulation is on XILINX software 14.3. Further, the performance metrics of multipliers such as area and delay are determined and compared.

In Dhole et al. [6] describe about the design of three Vedic multipliers using Vedic mathematics that help in reducing overall power and delay. Xilinx 14.7 software is used for the simulation. The report consists of the delay comparison too.

Here, there is a mention about the sutras [7].

From [8], [9] and [10] references on Vedic multiplication are taken.

This book provides a broad coverage of Verilog HDL and has practical designs of Verilog [11].

The DE10-lite system CD contains the documentation and materials such as the user manual, reference designs, and datasheets of the device. User manual provides the complete reference for the FPGA [12].

3 Vedic Mathematics

Vedic mathematics [1] is a primitive method of mathematics of India, which was developed 2500 years ago. "Vedic" comes from "Veda" meaning knowledge. Vedic mathematics has arithmetic rules that allow faster implementation. The sixteen principles called as sutras are the main basis of Vedic mathematics [7]. These are very attractive, and they give effective algorithms which can be adapted in several areas of engineering.

3.1 Urdhwa-Tiryakbhyam

The multiplier is based on an algorithm "Urdhwa-Tiryakbhyam" of the ancient Indian Vedic mathematics. "Urdhwa" and "Tiryakbhyam" words are derived from Sanskrit. This principle is called the "crowning gem of Vedic mathematics" [1], because it is the ultimate technique; we can multiply any number into any number within a single line and in less time.

The following example is based on a generic formula that can be used for all cases of multiplication involving large numbers.

Example 1: 88 × 97.

1. Subtract 100–88 - > 12 and 100–97 - > 3

2. Subtract crosswise to get equal results: 88 – 3 = 97 – 12 = 85, the first figure of the answer is 85.

3. Multiply vertically: 12 × 3 to Get 36, the last figure of the answer. Hence, 88 × 97 = 8536 (Fig. 1).

Some conventional methods used for multiplication operation, such as Booth's method are not very efficient in speed and number of elements. In the conventional method, partial products are added only after every partial product is obtained. In Vedic principle, as shown in the figure, partial products are calculated vertically, and once all the elements of a column are obtained, respective partial products are added simultaneously. Hence, speed advantage is obtained over the conventional method. This method can be extended to higher bits like 16, 32, 64 multipliers, and their performance can be compared. The hierarchical structure and the parallel addition of the partial product terms play a prominent role in increasing the speed of Vedic multiplier. This design hence saves resources in FPGA for all applications.

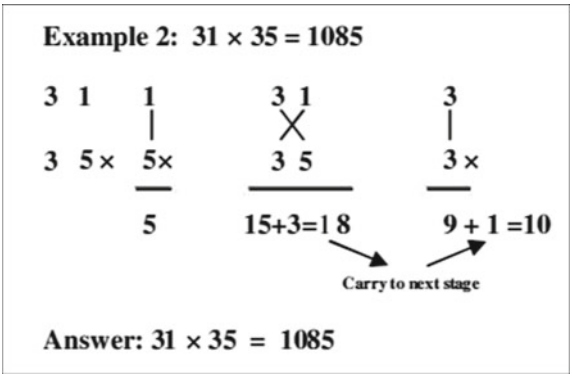


Fig. 1 Example of Vedic multiplication [8]

4 Implementation of Urdhwa-Tiryabhyam (Algorithm)

Based on the "Urdhwa-Tiryakbhyam" Sutra, an 8×8 bit multiplier is designed as follows.

The basic module is a 2-bit multiplier as per Vedic principle.

Considering a 2×2 bit multiplication using Vedic methodology,

Take $a = 01$ and $b = 11$ (Fig. 2).

(1) AND operation of a [0] with b [0]
(2) AND operation of a [0] with b [1]—(Cross)
(3) AND operation of a [1] with b [0]—(Cross)
(4) AND operation of b [1] with a [0]—(Cross)

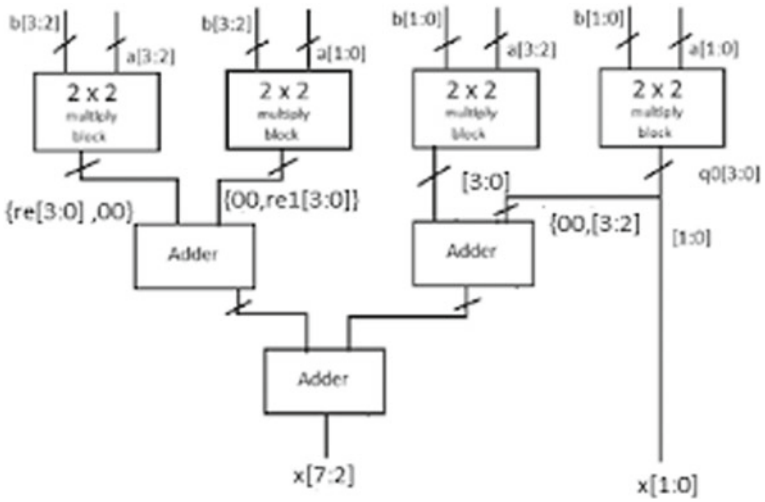


Fig. 2 Block diagram of a 2×2 Vedic multiplier [12]

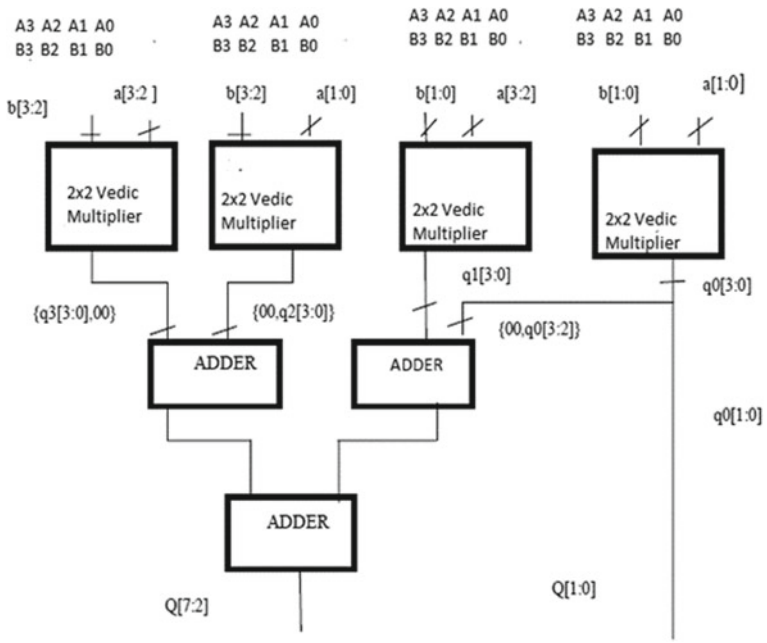


Fig. 3 Block diagram of a 4×4 Vedic multiplier [9]

(5) ADD the above side by side

Similarly, the next block is a 4×4 bit multiplier.

Example: **a = 1100** and **b = 1001** (Fig. 3.

(1) 2 × 2 multiplier for a[3:2] and b[3:2] then concatenate 00; total bits are now 6 bits
(2) 2 × 2 multiplier for a[1:0] and b[3:2] then concatenate 00; total bits are now 6 bits
(3) Add the results of step 1 and step 2
(4) 2 × 2 multiplication of a[3:2] and b[1:0]
(5) 2 × 2 multiplication of a[1:0] and b[1:0]
(6) Let the result of 6th step be q, then concatenate 00 before q[3:2]
(7) Now, adding the results of steps 6 and 5
(8) Then, adding the results of steps 7 and 3
(9) x[7:2] is the result.
(10) and x[1:0] is same as q[1:0]
(11) Concatenating the above two, we get the result x[7:0]

8-bit Vedic multiplier consists of both 4 and 2 bit modules.

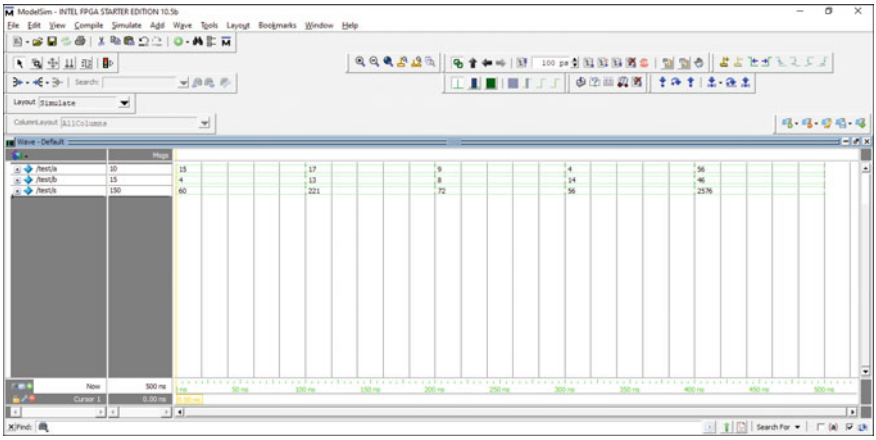


Fig. 4 Testbench simulation results of the 8×8-bit Vedic multiplier

5 Software and Hardware Implementation and Results

The code of the 8×8 bit Vedic multiplier is written in Verilog HDL. Modelsim-Altera 10.5b is used in conjunction with Intel Quartus Prime (18.1) tool for simulation and logic synthesis. Modelsim (Mentor Graphics) is used for simulating Verilog HDL (Hardware Description Languages). The testbench simulation results of an 8-bit multiplier are as in Fig. 4.

To verify the results, a 4-bit multiplier is implemented and tested on a DE10-lite FPGA board. The inputs are two 4-bit numbers. Input A (SW3 SW2 SW1 SW0) is displayed on the HEX0 location of the seven segment display. Input B (SW7 SW6 SW5 SW4) is displayed on the HEX2 location of the seven segment display. The result is displayed on the HEX4 and HEX5 locations. The inputs and outputs are displayed as in Fig. 5.

6 Comparison with Booth's Multiplier

For comparing the performance of the Vedic multiplier, a multiplier based on Booth's algorithm Fig. 6 is simulated and verified for various parameters.

The testbench simulation results of the Booth's multiplier [4] is as in Fig. 7.

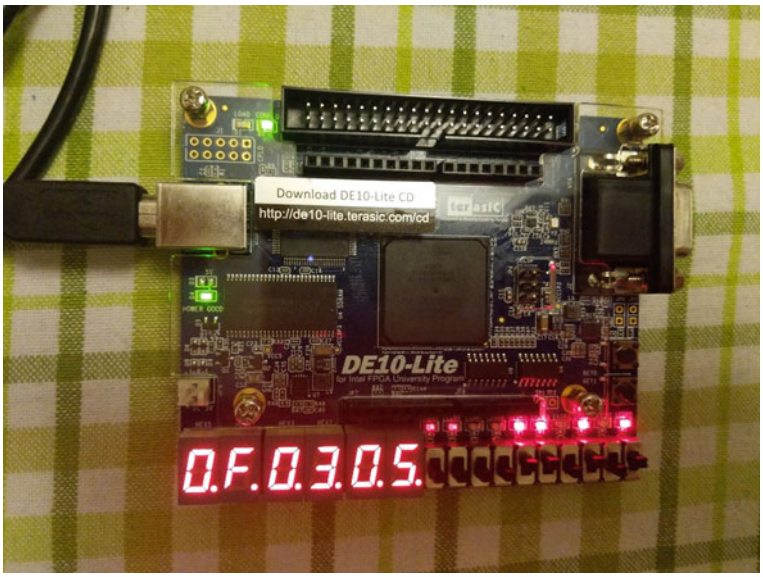
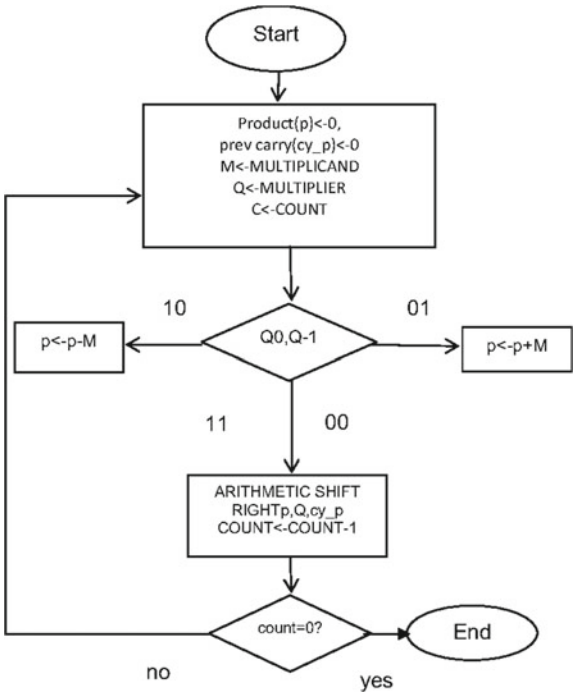


Fig. 5 Inputs and output on a DE10-lite board

Fig. 6 Booth's algorithm [4]



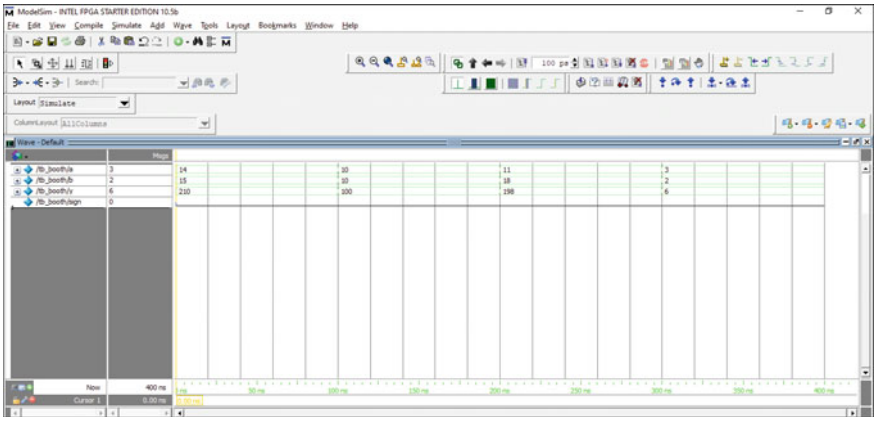


Fig. 7 Testbench results of the Booth's multiplier [4]

6.1 Logic Elements

The number of LE's used in the Vedic multiplier is ***161***, whereas the LE's used in the Booth multiplier is ***184***.

The "Analysis and Synthesis" Report of the 8×8 Vedic multiplier gives the flow summary as in Fig. 8.

The "Analysis and Synthesis" Report of the Booth multiplier gives the flow summary as in Fig. 9.

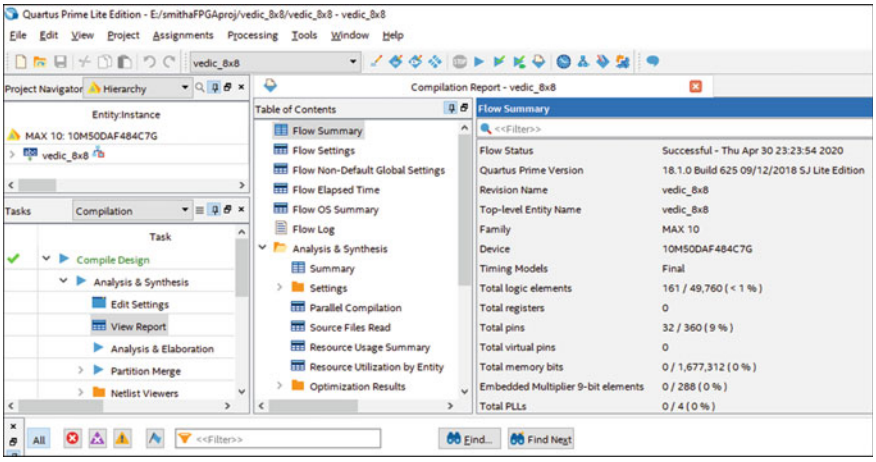


Fig. 8 Flow summary of the 8×8 Vedic multiplier

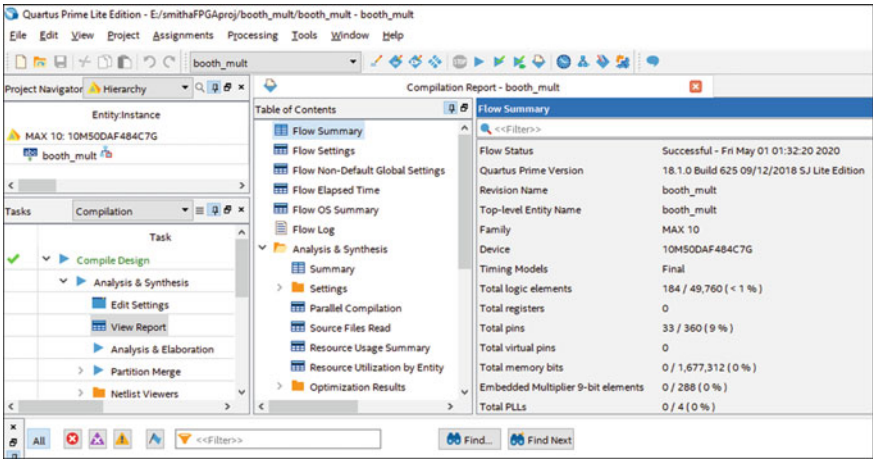


Fig. 9 Flow summary of the booth multiplier

6.2 *Propagation Delay*

The timing analyzer provides the minimum propagation delay reports of the 8×8 Vedic multiplier and the Booth's multiplier as below. The values are lesser in the Vedic multiplier.

Propagation delay—is the longest delay between the edges of a signal propagating from an input port to an output port. It is in nanoseconds (Figs. 10 and 11).

- RR—rising edge to rising edge longest delay.
- RF—rising edge to falling edge longest delay.
- FR—falling edge to rising edge longest delay.
- FF—falling edge to falling edge longest delay.

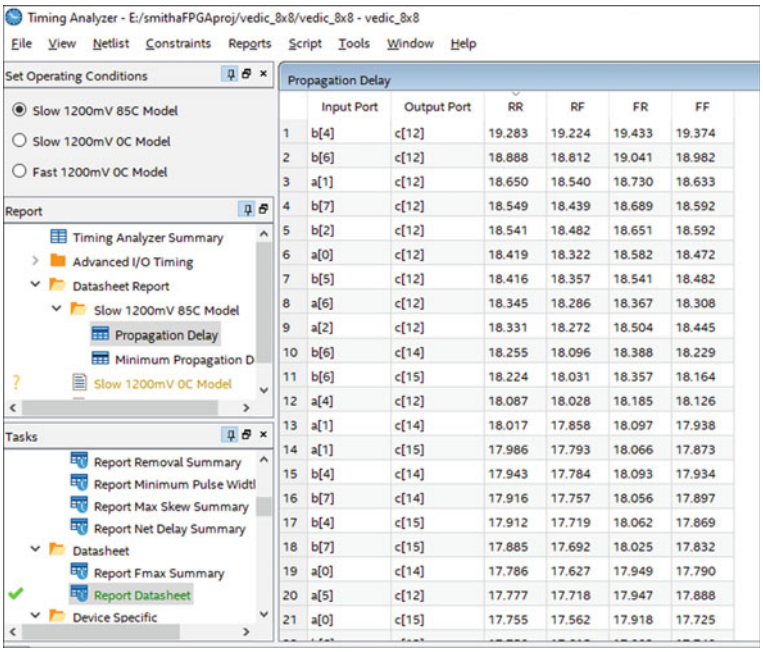
6.3 *Structure*

The RTL viewer diagram of both the multipliers clearly indicates that Vedic multiplier is more structured. Figures 12 and 13 show the various RTL viewer diagrams of the Vedic multiplier and the Booth's multiplier.

6.4 *Power Analyzer Summary*

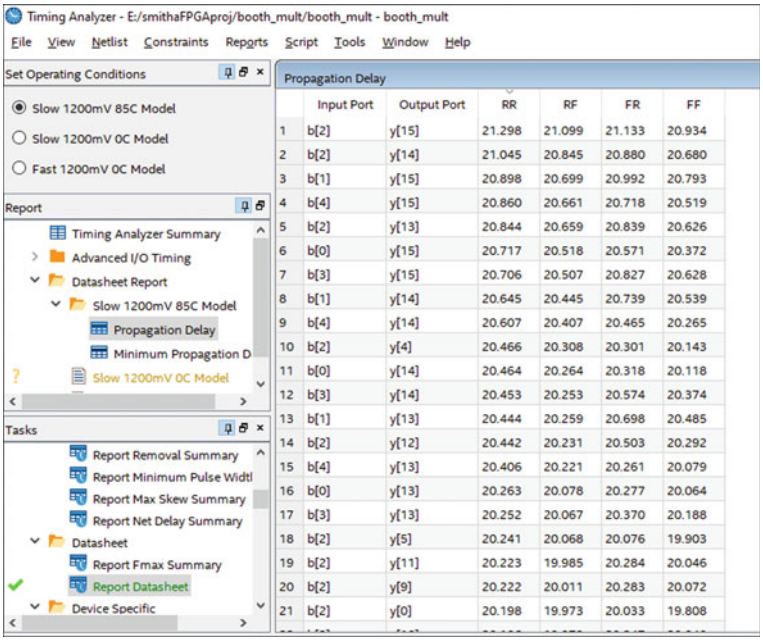
Both multipliers show similar power dissipation (Fig. 14).

The below Table 1 comprises of the comparison of the various performance parameters of both the multipliers:



	Input Port	Output Port	RR	RF	FR	FF
1	b[4]	c[12]	19.283	19.224	19.433	19.374
2	b[6]	c[12]	18.888	18.812	19.041	18.982
3	a[1]	c[12]	18.650	18.540	18.730	18.633
4	b[7]	c[12]	18.549	18.439	18.689	18.592
5	b[2]	c[12]	18.541	18.482	18.651	18.592
6	a[0]	c[12]	18.419	18.322	18.582	18.472
7	b[5]	c[12]	18.416	18.357	18.541	18.482
8	a[6]	c[12]	18.345	18.286	18.367	18.308
9	a[2]	c[12]	18.331	18.272	18.504	18.445
10	b[6]	c[14]	18.255	18.096	18.388	18.229
11	b[6]	c[15]	18.224	18.031	18.357	18.164
12	a[4]	c[12]	18.087	18.028	18.185	18.126
13	a[1]	c[14]	18.017	17.858	18.097	17.938
14	a[1]	c[15]	17.986	17.793	18.066	17.873
15	b[4]	c[14]	17.943	17.784	18.093	17.934
16	b[7]	c[14]	17.916	17.757	18.056	17.897
17	b[4]	c[15]	17.912	17.719	18.062	17.869
18	b[7]	c[15]	17.885	17.692	18.025	17.832
19	a[0]	c[14]	17.786	17.627	17.949	17.790
20	a[5]	c[12]	17.777	17.718	17.947	17.888
21	a[0]	c[15]	17.755	17.562	17.918	17.725

Fig. 10 Propagation delay report of the Vedic multiplier



	Input Port	Output Port	RR	RF	FR	FF
1	b[2]	y[15]	21.298	21.099	21.133	20.934
2	b[2]	y[14]	21.045	20.845	20.880	20.680
3	b[1]	y[15]	20.898	20.699	20.992	20.793
4	b[4]	y[15]	20.860	20.661	20.718	20.519
5	b[2]	y[13]	20.844	20.659	20.839	20.626
6	b[0]	y[15]	20.717	20.518	20.571	20.372
7	b[3]	y[15]	20.706	20.507	20.827	20.628
8	b[1]	y[14]	20.645	20.445	20.739	20.539
9	b[4]	y[14]	20.607	20.407	20.465	20.265
10	b[2]	y[4]	20.466	20.308	20.301	20.143
11	b[0]	y[14]	20.464	20.264	20.318	20.118
12	b[3]	y[14]	20.453	20.253	20.574	20.374
13	b[1]	y[13]	20.444	20.259	20.698	20.485
14	b[2]	y[12]	20.442	20.231	20.503	20.292
15	b[4]	y[13]	20.406	20.221	20.261	20.079
16	b[0]	y[13]	20.263	20.078	20.277	20.064
17	b[3]	y[13]	20.252	20.067	20.370	20.188
18	b[2]	y[5]	20.241	20.068	20.076	19.903
19	b[2]	y[11]	20.223	19.985	20.284	20.046
20	b[2]	y[9]	20.222	20.011	20.283	20.072
21	b[2]	y[0]	20.198	19.973	20.033	19.808

Fig. 11 Propagation delay report of the Booth's multiplier

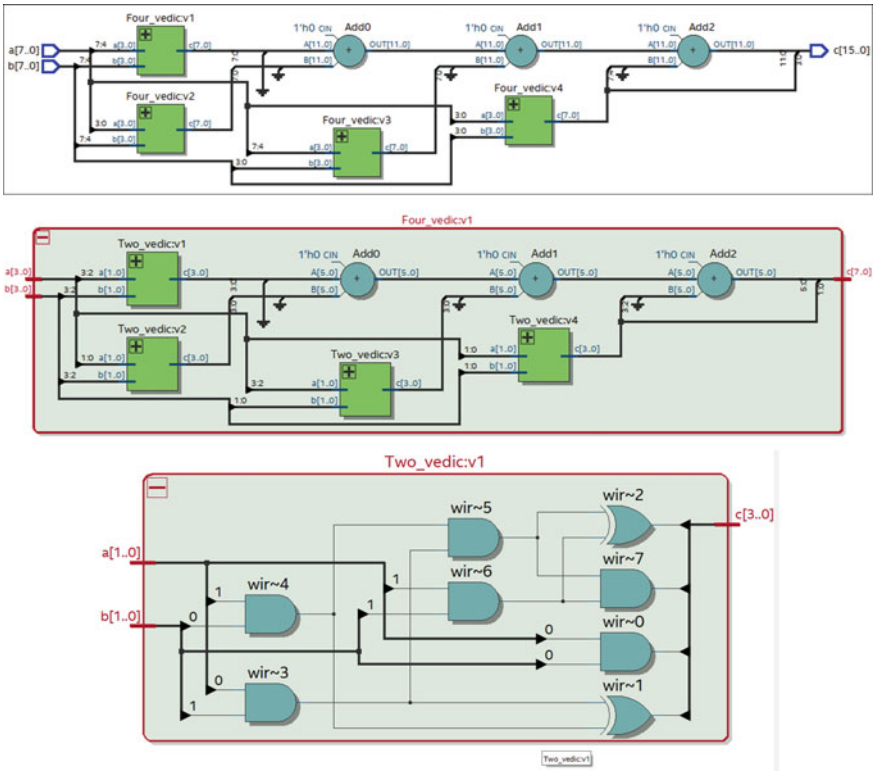


Fig. 12 RTL structure of the Vedic multiplier

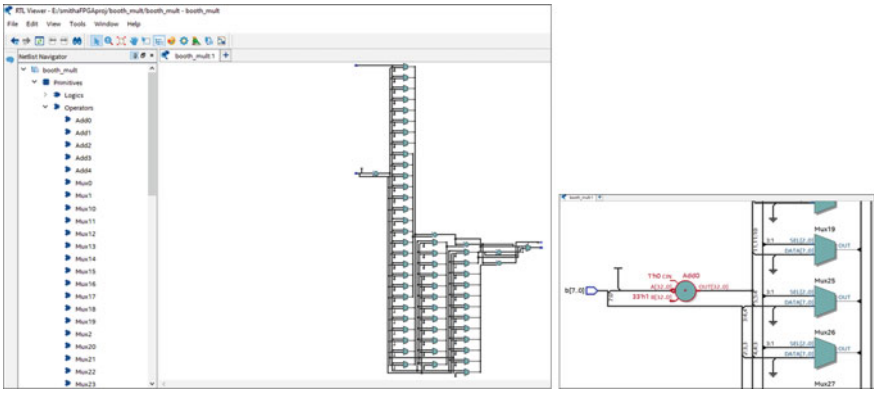


Fig. 13 RTL structure of the Booth's multiplier

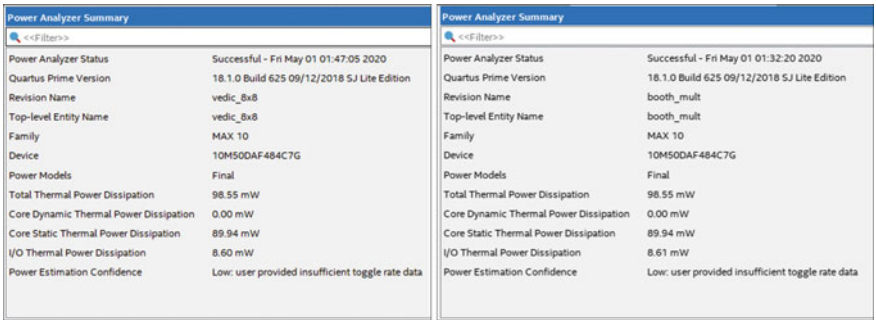


Fig. 14 Power analyzer summary diagrams of Vedic and Booth's multipliers

Table 1 Performance comparison table of the Vedic and Booth's multipliers

Sr. No.	Parameters	Vedic multiplier	Booth's multiplier
1	Number of LE's	161	184
2	Worst propagation delay	19.433 ns	21.298 ns
3	Thermal power dissipation	98.55 mW	98.55 mW
4	RTL structure	Structured	Not structured

7 Conclusion

An 8×8-bit multiplier is successfully simulated and implemented using Vedic techniques. It is compared with an 8×8-bit Booth's multiplier. Based on the simulation, FPGA outputs and performance analysis arrived at a conclusion that the vedic multiplier is more structured, uses lesser logical elements, has lesser propagation delays and hence faster. The performance would be even more evident and more beneficial in Vedic multipliers of further higher bits like 16×16, 32×32, and so on.

References

1. Maharaja, B.K.T.: Vedic Mathematics, Motilal Banarsidass Publishers (1965)
2. Johny, N.: Area and Delay efficient Multiplier using Vedic Mathematics. New Horizon College of Engineering, Bangalore, Karnataka, India (2016)
3. Gadakh, S.N., Khade, A.A.: FPGA implementation of high speed Vedic multiplier. Int. Conf. Workshop Electron. Telecommun. Eng. ICWET Mumbai 184–187 (2016). https://doi.org/10.1049/cp.2016.1144

4. Sunitha, G.S., Rakesh, H.M.: Performance Comparison of Conventional Multiplier with Vedic Multiplier Using ISE Simulator. Bapuji Institute of Engineering and Technology, Davanagere, Karnataka, India (2018)
5. Ingle, A., Oza, Dr.: FPGA Implementation of Novel High Speed Multiplier. BVDU's college of Engineering, Pune, Maharashtra, India (2015)
6. Dhole, S., Shembalkar, S., Yadav, T.: Design and FPGA Implementation of 4 × 4 Vedic Multiplier using Different Architectures. RCOEM, Nagpur, India (2017)
7. Y. Bansal, C.M., Kaur, P.: High speed vedic multiplier designs-a review. In: 2014 Recent Advances in Engineering and Computational Sciences (RAECS), Chandigarh (2014)
8. Sriraman, L., Prabakar, T.N.: Design and Implementation of Two variable Multiplier using KCM and Vedic Mathematics. Oxford Engneering College, Trichy, Tamilnadu, India (2012)
9. Dheeksha S.S., Kiran Kumar V.G.: VLSI Implementation of Arithmetic Operation, Sahyadri College of Engneering and Management, India (2016)
10. Magesh, V., Selvaoviya, S., Soundarya, B., Pooja, S.V.: FPGA implementation of pipelined FIR filter using vedic multiplier. Int. J Appl. Eng. Res. **14**(6), ISSN 0973-4562 (2019)
11. Samir, P.: Verilog HDL—A guide to Digital Design and Synthesis, 2nd edn. Dorling Kindersley and Pearson Education, Inc (2008)
12. www.intel.com DE10-Lite System CD and DE-10 Lite Manual.